

Banking Web Application Simulator

Documentation

Overview

This document describes a banking web application simulator built with React (customer and admin portals), FastAPI backend, and PostgreSQL. All banking operations are simulated and controlled through an Admin Panel.

Technology Stack

Frontend: React, TypeScript, Vite, Tailwind CSS, React Router, TanStack Query. Backend: FastAPI, SQLAlchemy, Alembic, JWT, Pydantic. Database: PostgreSQL. Optional: Redis, Celery. Deployment: Docker, Nginx, Uvicorn.

Customer Features

Authentication, Dashboard, Account Management, Transaction History, Simulated Transfers, Card Management, Loan Services, Savings & Investments, Cheque Services, Notifications, Customer Support, Profile Management, Document Center, Foreign Exchange, Security, KYC, Responsive UI, Search, Multilingual Support, Accessibility, PDF generation.

Admin Panel

Create users/accounts, create and edit transactions, change transaction status (pending/successful/failed/etc.), edit balances, manage cards, loans, savings, notifications, FX rates, documents, KYC approvals, audit logs, role-based access.

Database

Core entities: Users, Accounts, Transactions, Cards, Loans, SavingsAccounts, InvestmentAccounts, ChequeRequests, Notifications, Documents, KYC, SupportTickets, Sessions, Devices, AdminUsers, AuditLogs, FXRates.

Development Roadmap

1. Backend setup. 2. Authentication. 3. Customer portal. 4. Account linking. 5. Transfers. 6. Admin portal. 7. Admin operations. 8. Cards/Loans/Savings/KYC. 9. Notifications/Documents/FX. 10. Testing, Docker deployment, custom domain.